

Mirrorwalk: An Outdoor Augmented-Reality Ghost-Race Running Application

Project Proposal for UCS503P

Submitted to: Dr. Sourav Roy

Thapar Institute of Engineering and Technology

Ana Hasan, CSED*

Agrim Gupta, CSED[†]

Janvi Verma, CSED[‡]

Yashkaran Singh, CSED[§]

August 20, 2026

1 Higher-order goal

... secondary framing

This project aims to advance the state of consumer-grade outdoor augmented reality by tackling a problem most fitness applications avoid: keeping a virtual object spatially anchored to the real world across multiple sessions, outdoors, without markers. While the immediate deliverable is a running application, the underlying engineering objective is to demonstrate that *persistent outdoor AR localization* is achievable within the accuracy and resource constraints of a consumer smartphone.

2 Time-to-value

... primary heuristic

- We will deliver a working, demoable increment early by first proving out AR ghost rendering on a short, fixed loop before attempting cross-session persistence.
- We will prioritize a single reliable outdoor route over broad feature coverage, so that every iteration produces something that can be tested in the field, not just in a simulator.
- Success in the first iteration is defined narrowly: a recorded run can be replayed as a spatially anchored ghost at the same physical location in a later session.

3 Problem Statement

Existing running applications (e.g. Nike Run Club, Strava) offer “ghost pacing” only as numeric or audio feedback — a pace number, a beep when you fall behind. None render the ghost as a spatially present object in the runner’s actual environment. This gap exists for a concrete technical reason, not a product oversight:

- **No reliable outdoor anchoring:** Raw GPS accuracy (3–10 m, worse near buildings) is far too coarse to visually lock a virtual object to a specific point on a street.

*Roll No: 1024160103, Email: ahasan_be24thapar.edu

[†]Roll No: 1024160117, Email: agupta_be24thapar.edu

[‡]Roll No: 1024160108, Email: jverma_be24thapar.edu

[§]Roll No: 1024160125, Email: ysingh_be24thapar.edu

- **Session discontinuity:** AR frameworks (ARKit/ARCore) track pose accurately *within* a session using visual-inertial odometry (VIO), but that coordinate space does not persist — the same physical location has no shared reference across two separate app launches.
- **Perceptual fragility:** even small pose drift is highly noticeable to a user, since a ghost that “teleports” or drifts off the sidewalk breaks the illusion immediately, unlike drift in a non-visual pacing metric.

Why this matters now (contextual relevance): Outdoor visual positioning services (VPS), such as ARCore’s Geospatial API, have only recently become accurate and available enough for a student team to build against directly, rather than needing to implement positioning research from scratch.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **native Android** application, Mirrorwalk, with the following components:

- **Route recording:** capture a run’s path (GPS + timestamped pose) on a short, fixed outdoor loop.
- **Ghost replay:** render a translucent 3D runner mesh that retraces the recorded route in AR, timed to the original pace.
- **Outdoor anchoring:** use ARCore’s Geospatial API to keep the ghost spatially locked to the correct street position across separate sessions, rather than relying on GPS or a hand-built sensor filter alone.
- **Post-run analytics:** splits and pace comparison between the live run and the ghost.

4.2 Core workflow

1. Runner records a run on the chosen loop; the route and per-point timing are stored locally.
2. On a later run of the same loop, the app initializes an AR session, resolves the runner’s Geospatial pose, and spawns the ghost mesh at the recorded start point.
3. The ghost advances along the stored route at the recorded pace while the runner moves through the real environment, visible through the camera.
4. After the run, the app displays splits and a pace-delta comparison against the ghost.

4.3 Operational constraints for an educational, 5-week setting

- Scope anchoring and rendering to a single short loop (a few hundred metres), not multi-kilometre routes, to keep drift and testing time bounded.
- Rely on ARCore’s built-in VIO and Geospatial API for pose estimation rather than implementing a custom Kalman filter from scratch, since the team has no prior AR experience.
- Design for local-only data storage; no backend or multi-device sync is required for the MVP.

5 Solution Approach

... Engineering focus

This is an engineering course project, so we focus on correctly integrating and evaluating an existing outdoor-AR positioning system, rather than researching new localization algorithms.

5.1 Anchoring strategy

... *non-research*

Outdoor persistence will be achieved via integration, not novel algorithm design:

- Use ARCore’s Geospatial API as the primary source of outdoor pose, since it already fuses VPS, GPS, and VIO internally.
- Use raw GPS only for coarse route logging and initial session framing, not for final anchoring.
- Where Geospatial accuracy is degraded (e.g. VPS coverage gaps), fall back gracefully to a shorter or indoor-adjacent test loop rather than presenting an unstable ghost.

5.2 App architecture

... *native Android*

A single-platform native architecture:

- **AR/Sensor layer:** Kotlin, ARCore + Geospatial API, SceneView for mesh rendering.
- **Application layer:** Jetpack Compose UI, Kotlin Coroutines/Flow for async sensor and AR session state.
- **Local storage:** Room (SQLite) for runs, routes, and splits.

5.3 Iterative delivery enabler

Weekly, field-testable milestones will be used to:

- Validate AR rendering and anchoring incrementally (indoor test → short outdoor loop → cross-session replay), rather than integrating all pieces at once.
- Catch drift, coverage, and performance issues early, since these are the project’s primary technical risks.
- Keep each week’s output demoable, even if scope has to be trimmed later.

6 Evaluation Criterion

... *measurable and attributable*

We define success using a combination of technical accuracy and user-experience metrics, since AR quality is ultimately judged both by measurable drift and by whether a user perceives the ghost as stable.

6.1 Primary evaluation metrics

- **Anchoring drift:** horizontal offset (in metres) between the ghost’s rendered position and its intended position along the recorded route, sampled at fixed intervals during replay.
- **Session-to-session consistency:** variance in the ghost’s start-position anchor across repeated sessions at the same physical location.

6.2 Secondary evaluation metrics

- **Perceived stability:** tester-reported rating (1–5) of whether the ghost appeared to stay “on the ground/path” versus drifting or jittering.
- **Session initialization time:** time from app launch to a usable AR anchor being resolved.
- **Reliability:** proportion of test sessions in which the ghost renders and completes replay without crashing or losing tracking.

6.3 Pilot validation plan

... *high-level*

- Record and replay on a single fixed outdoor loop near campus, repeated across at least 5–10 sessions at different times of day.
- Recruit 3–5 team members and peers as test runners to rate perceived stability and usability.
- Compare drift and consistency metrics across sessions to characterize how much Geospatial accuracy varies in the test location.

7 Scalability

... *with foresight*

The proposition should scale in both theoretical and practical senses, even though the MVP is deliberately scoped to one loop.

1. **Scalable (theoretically):** The system should extend to more routes and longer distances by:
 - keeping route/run storage schema-generic (not hard-coded to one loop),
 - isolating the AR anchoring module so it can be re-tuned per route without touching UI or analytics code,
 - structuring splits/analytics to work over routes of arbitrary length.
2. **Operationally deployable (practically):** The system should remain installable and testable without infrastructure:
 - fully local data storage for the MVP, with a clear extension point for cloud sync later,
 - no backend dependency for core functionality,
 - straightforward APK distribution for testing.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a native Android AR project:

- ARCore and its Geospatial API for outdoor visual positioning.
- SceneView for AR mesh rendering without hand-written OpenGL.
- Jetpack Compose for UI, and Room for local persistence.
- Google Play Services FusedLocationProviderClient for GPS logging.
- Standard Android instrumentation/unit testing for non-AR logic.

We will use these mature components to focus on correct integration, field testing, and measurement of AR stability, rather than inventing new positioning infrastructure.

9 Project scope and deliverables

... *iteration-friendly*

9.1 Initial deliverable

... *first iteration*

- Route recording (GPS + timestamps) and local storage
- Basic AR ghost rendering, tested indoors/short range
- Single fixed outdoor loop with Geospatial-anchored ghost replay
- Splits screen with pace comparison against the ghost
- Logging instrumentation for drift and session-consistency measurement

9.2 Subsequent deliverables

- Graceful degradation when Geospatial/VPS accuracy is poor (e.g. shrink or flag the route rather than render an unstable ghost)
- Support for a second, different outdoor loop to test generality of the anchoring approach
- Basic run-history and best-time tracking UI
- Stretch: dead-reckoning fallback during brief GPS/VPS dropout

10 Risks and mitigations

- **No prior AR experience on the team:** timebox a small “hello-world” ARCore/Geospatial spike in the first 2–3 days to surface the learning curve early, before committing to the full schedule.
- **VPS coverage gaps at the test location:** scout and confirm Geospatial coverage quality at the intended test loop before finalizing route choice.
- **Perceptual drift undermining the demo:** keep the MVP route short and re-test frequently, rather than discovering drift issues only at the multi-week mark.
- **Schedule risk from AR debugging (historically underestimated):** reserve the final week explicitly as buffer for AR-specific bugs rather than new features.

11 Summary

This project proposes a native Android AR application that lets a runner race a spatially anchored ghost of their own past run, addressing a gap that existing fitness apps avoid because outdoor AR anchoring is genuinely hard. It meets the course criteria by emphasizing time-to-value through incremental, field-testable iterations; using a deliberately scoped MVP (single short loop) to keep risk bounded within five weeks; and defining measurable evaluation criteria spanning both technical accuracy (anchoring drift, session consistency) and user experience (perceived stability, reliability). It remains focused on correct engineering integration of existing outdoor-AR infrastructure rather than research-driven novelty in positioning algorithms.